

Python Static Security Code Analyzer

Prepared by Students of
Software Engineering Department

Name	ID
Mai Ashraf	192300095
Malak Mohamed	192300094
Gana Ahmed	192300067
Basma yasser	192300538
Menna allah mostafa	192300166

Supervised by: Dr. Hossam Reda
TA. Asmaa Boghdady \ Mohamed khtab

Contents

1. Introduction	2
2- Project Vision	3
3. System Architecture	4
5. How the System Works	5
5.1 Input Handling.....	5
5.2 File Processing	5
5.3 AST (Abstract Syntax Tree) Conversion	6
5.4 Code Traversal Engine.....	6
6. Security Rule Engine	6
6.1 Dangerous Function Detection	7
6.2 System Command Execution Detection	7
6.3 Hardcoded Secrets Detection.....	8
6.4 Unused Variable Detection.....	8
8. Output System	9
Terminal Output.....	9
File Report Generation.....	9
Example Output.....	10
9. Key Concepts Demonstrated	10
10. Security Significance.....	11
11. Conclusion.....	11

1. Introduction

In modern software development, security has become one of the most critical aspects of building reliable and robust applications. Software vulnerabilities can lead to severe risks such as data breaches, system compromise, and unauthorized execution of malicious code.

This project introduces a **Python-based Static Security Code Analyzer**, designed to examine Python source code without executing it, in order to identify potential security issues and improve code quality early in the development process.

The analyzer detects:

- **Potential security vulnerabilities**
- **Usage of dangerous functions**
- **Poor or risky coding practices**
- **Unused or redundant variables**

Unlike traditional runtime debugging tools, this system applies **Static Application Security Testing (SAST)** techniques. It analyzes the program structure using Python's **Abstract Syntax Tree (AST)** module, allowing deep inspection of code behavior at the syntax level.

The goal of this project is to help developers write safer and cleaner code by automatically identifying risky patterns before execution.

2- Project Vision

The main vision of this project is to simulate a lightweight version of real-world security tools used in modern software engineering environments. These include cybersecurity analysis tools, secure code review systems, and DevSecOps pipelines that are widely adopted in industry to ensure code reliability and security.

This project demonstrates how static analysis techniques can be applied early in the development lifecycle to identify risky coding patterns before execution. By doing so, it helps reduce potential vulnerabilities and encourages developers to follow safer coding practices.

Ultimately, the goal is to bridge the gap between academic learning and real-world security practices by providing a simplified yet practical implementation of a static code analysis system.

3. System Architecture

The system follows a modular pipeline architecture designed to ensure clear separation of concerns and maintainability throughout the analysis process.

The workflow consists of the following stages:

Input File → AST Parsing → Code Traversal → Rule Engine → Warning Generation → Report Output

- **Input File:** The system starts by accepting a Python source file as input for analysis.
- **AST Parsing:** The source code is parsed into an Abstract Syntax Tree (AST), which provides a structured representation of the code.
- **Code Traversal:** The AST is traversed node by node to inspect variables, function calls, and operations.
- **Rule Engine:** A set of predefined security rules is applied to detect suspicious or unsafe patterns in the code.
- **Warning Generation:** Detected issues are categorized based on severity levels such as HIGH, MEDIUM, and LOW.
- **Report Output:** Finally, a structured report is generated and displayed in the terminal and saved to an external file.

This modular design allows each component to operate independently, making the system easier to extend, debug, and enhance with additional security rules in the future.

4. Core Modules & Libraries

The system is implemented using Python's built-in libraries, which ensures lightweight execution without external dependencies and improves portability.

◇ Python Built-in Modules Used:

- **ast (Abstract Syntax Tree):**
This is the core module of the project. It is used to parse Python source code into a structured tree representation, allowing safe and systematic analysis of code without execution.
 - **sys:**
Used to handle command-line arguments, enabling the user to pass the target file dynamically when running the scanner.
 - **time:**
Used to generate timestamps in the security report, providing traceability for when the analysis was performed.
 - **builtins:**
Used to identify Python's built-in functions and exclude them when detecting unused variables, improving the accuracy of analysis.
-

5. How the System Works

The system operates through a structured analysis pipeline that processes Python source code step by step, transforming it from raw text into meaningful security insights.

5.1 Input Handling



The tool begins by accepting a Python source file through the command line interface:

If no file is provided, the program terminates gracefully and displays usage instructions. This ensures controlled execution and prevents unexpected runtime errors.

5.2 File Processing

The selected file is securely opened using UTF-8 encoding with error handling enabled. This guarantees that the system can handle files with non-standard or corrupted characters without crashing, improving overall robustness.

5.3 AST (Abstract Syntax Tree) Conversion

```
tree = ast.parse(code)
```

The raw source code is parsed into an Abstract Syntax Tree (AST) using:

This transformation is a critical step, as it converts the code into a structured hierarchical representation. Instead of analyzing plain text, the system now works with meaningful syntactic elements such as assignments, function calls, and expressions.

If a syntax error exists in the source code, the parsing process is halted immediately, and the error is reported to ensure early detection of invalid code.

5.4 Code Traversal Engine

```
ast.walk(tree)
```

Once the AST is generated, the system performs a full traversal of all nodes using:

This traversal ensures comprehensive coverage of the entire code structure. During this phase, the analyzer inspects:

- Variable assignments
- Function calls
- Variable usage patterns

- Attribute access operations

Each node is evaluated against a predefined set of security rules to identify potentially unsafe or suspicious behavior.

6. Security Rule Engine

The Security Rule Engine is the core component of the system responsible for analyzing the Abstract Syntax Tree (AST) and applying a set of predefined rules to detect potentially unsafe or suspicious code patterns.

Each rule targets a specific category of security risk, allowing structured and systematic detection.

6.1 Dangerous Function Detection

The system identifies the usage of high-risk functions such as:

- `eval()`
- `exec()`

These functions are considered dangerous because they allow the execution of dynamically generated code at runtime.

Security Risks:

- Code injection attacks
- Execution of untrusted or malicious payloads
- Potential full system compromise if misused

6.2 System Command Execution Detection

The engine detects direct system-level calls including:

- `os.system()`

- `subprocess.call()`

These functions enable interaction with the underlying operating system.

Security Risks:

- Unauthorized execution of system commands
- File manipulation or deletion
- Execution of external or malicious scripts

6.3 Hardcoded Secrets Detection

```
password = "123456"
```

The analyzer detects sensitive information embedded directly in the source code, such as:

Security Risks:

- Exposure of sensitive credentials
- Increased risk if source code is leaked or shared
- Violation of secure coding best practices

6.4 Unused Variable Detection

The system tracks variable lifecycle by comparing:

- Variables that are assigned values
- Variables that are actually used in the code

```
Unused = Assigned - Used
```

It then computes unused variables using:

Purpose:

- Improve code quality and readability
 - Reduce unnecessary memory usage
 - Detect redundant or incomplete logic
-

7. Warning Classification System

level	Meaning	Impact
High	Critical security risk	System compromise possible
Medium	Moderate risk	Security concern
Low	Code quality issue	Non-critical improvement

8. Output System

The Output System is responsible for presenting the results of the analysis in a clear, structured, and user-friendly format. It ensures that detected security issues are easily interpretable by developers.

Terminal Output

The system displays real-time results in the terminal using ANSI escape codes to enhance readability through color-coded severity levels:

- **Red → High Risk**
- **Yellow → Medium Risk**
- **Green → Low Risk**

This visual classification allows developers to quickly identify and prioritize critical issues.

File Report Generation

report.txt

In addition to terminal output, the system generates a persistent report saved as:

The report includes structured information such as:

- Severity level
- Issue type
- Line number
- Detailed description of the issue
- Timestamp of analysis

This ensures traceability and allows offline review of security findings.

Example Output

[HIGH] Dangerous function os.system() at line 12

[MEDIUM] Hardcoded Password at line 8

[LOW] Unused variable: temp (defined at line 20)

===== END REPORT =====

This format provides a clear hierarchical view of issues based on severity, making it easier to prioritize fixes.

9. Key Concepts Demonstrated

This project demonstrates several fundamental and advanced software engineering concepts, including:

- Abstract Syntax Tree (AST) based static analysis
- Static Application Security Testing (SAST) principles

- Security vulnerability detection techniques
- Efficient use of data structures (sets, dictionaries)
- Command-Line Interface (CLI) application design
- File handling and encoding management
- Pattern recognition and rule-based logic

These concepts collectively simulate real-world security analysis tools used in professional DevSecOps environments.

10. Security Significance

This tool mimics real-world security systems used in:

- **DevSecOps pipelines**
- **Code review automation tools**
- **Static Application Security Testing (SAST) tools**

It helps detect vulnerabilities before code execution, reducing security risks significantly.

11. Conclusion

This project presents a foundational implementation of a Python-based static security analysis tool designed to inspect source code without execution. It demonstrates how structural code analysis can be leveraged to identify potential vulnerabilities, improve software quality, and encourage secure coding practices.

By utilizing Abstract Syntax Tree (AST) analysis and a rule-based detection engine, the system successfully simulates core principles of Static Application Security Testing (SAST). This highlights the importance of analyzing code at an early stage in the development lifecycle to prevent security issues before runtime.

Although simplified compared to industrial-grade security solutions, this project provides a strong conceptual foundation and serves as a stepping stone toward more advanced cybersecurity and DevSecOps tools used in real-world environments.

